

Grounded Reasoning: Why Embedding an LLM in an Executable Environment Improves Performance

May 2026

The Problem with Pure Reasoning

Large language models (LLMs) are trained to predict plausible continuations of text. This produces models that are remarkably fluent and knowledgeable, but it carries a fundamental limitation: the model’s “beliefs” about the world are encoded as distributional biases in its weights, and there is no mechanism by which incorrect reasoning is corrected by contact with reality. When an LLM reasons about code purely in natural language—thinking about what a program *will* do rather than running it—it generates whatever continuation is most likely under its training distribution. That distribution reflects the frequency with which descriptions appear in human text, not actual execution outcomes. The result is a systematic tendency toward plausible-sounding but incorrect reasoning: the model cannot reliably distinguish correct programs from subtly broken ones, because both appear in human-written discussion with similar surface characteristics.

Empirical Evidence for Execution Loops

The practical superiority of embedding LLMs in execution loops is now well-documented. Program-Aided Language Models (PAL) [1] demonstrated that directing an LLM to express reasoning steps as executable Python—and then running that code—substantially outperforms chain-of-thought prompting on mathematical and symbolic tasks. The mechanism is transparent: arithmetic and logical operations are delegated to the interpreter, which executes them exactly rather than approximately. ReAct [2] formalised this as a general framework, interleaving language reasoning traces with actions in an environment (including code execution). Crucially, ReAct outperformed both pure reasoning (no actions) and pure acting (no reasoning), suggesting the value lies in the combination of hypothesis formation and empirical testing. Reflexion [3] extended this further: agents that receive execution feedback and are prompted to reflect verbally on failures before retrying substantially outperform single-shot generation on programming benchmarks, constituting a lightweight form of iterative hypothesis refinement.

SWE-bench [4], a benchmark of real GitHub issues requiring code modification and test-suite verification, demonstrated that the gap between single-pass generation and iterative agents with execution feedback is large and practically significant. InterCode [5] treated code generation as a partially observable Markov decision process, with execution outcomes as observations, and found consistent gains across multiple programming environments.

Why Does It Work? Partial Theoretical Accounts

The empirical picture is clear; the theoretical understanding is less settled. Several complementary accounts have been offered, none fully satisfactory.

Grounding via unambiguous feedback. The most direct account is that execution provides feedback the model cannot hallucinate. A Python traceback is a deterministic output of a formal system—categorically different from the model’s self-evaluation, which is itself a text prediction subject to the same distributional biases that produced the error. The environment acts as an oracle external to the model’s uncertainty.

Offloading formal computation. PAL and related work suggest that LLMs are unreliable at tasks requiring exact computation—not from lack of relevant knowledge, but because approximating exact arithmetic through token prediction is inherently lossy. Generating code that delegates computation to the interpreter avoids attempting what the model does poorly. This mirrors what humans do when using a calculator rather than performing mental arithmetic for critical computations.

Error decomposition and localisation. In multi-step reasoning, errors compound: a single incorrect intermediate result propagates through subsequent steps. Execution grounds each step. If a program produces a wrong intermediate value, the test suite or downstream execution reveals this before compounding occurs. The loop converts a problem of unbounded error propagation into one of bounded, localised error correction.

On the Reservoir Computing Analogy

One might ask whether the programming runtime plays a role analogous to a reservoir in reservoir computing [6, 7]: a fixed, complex dynamical system that transforms inputs into a high-dimensional state space, with the LLM acting as a learned readout. The analogy is suggestive but should be treated with caution. In classical reservoir computing, the reservoir is a random, typically chaotic network whose high-dimensional nonlinear dynamics are exploited *precisely because* they are unlearned and fixed. The programming runtime, by contrast, is structured, deterministic, and semantically interpretable. More importantly, the LLM does not merely read from the runtime’s state; it generates instructions that determine what the runtime computes. The control flow runs in the opposite direction from standard reservoir computing.

A more defensible analogy is the extended mind hypothesis [8]: that cognitive processes can be distributed between an agent and its environment when the environment reliably stores and transforms information. On this view, the executing program is part of the reasoning process, not merely a tool external to it. Whether this framing is scientifically productive or merely a useful metaphor is itself an open question.

Honest Assessment and Open Questions

The core empirical finding—that LLMs in execution loops substantially outperform single-pass generation on coding and formal reasoning tasks—is robust and reproducible. The theoretical explanation is less settled. The grounding and offloading accounts are plausible and consistent with the data, but they do not constitute a mechanistic theory of why language model weights fail to reliably simulate formal computation internally. It remains unclear whether this is a fundamental limitation of autoregressive prediction as a training objective, a consequence of insufficient scale, or an architectural constraint that future models might overcome.

It is also worth being honest about what execution loops *cannot* do: they do not improve reasoning about problems that cannot be formalised or tested. The benefit is specific to domains with deterministic, observable outcomes. Finally, the question of whether empirical exploration in a formal environment is qualitatively different from internal reasoning—or merely more reliable—remains philosophically and empirically open. The honest summary is: we have strong evidence that it works, a plausible but incomplete story about why, and a significant theoretical gap remaining.

References

- [1] Gao, L. et al. PAL: Program-aided language models. *Proc. ICML*, PMLR 202:10764–10799, 2023.
- [2] Yao, S. et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [3] Shinn, N. et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [4] Jimenez, C. E. et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.
- [5] Yang, J. et al. InterCode: Standardizing and benchmarking interactive coding with execution feedback. *NeurIPS*, 2023.
- [6] Jaeger, H. The “echo state” approach to analysing and training recurrent neural networks. *GMD Report* 148, 2001.
- [7] Maass, W., Natschläger, T., Markram, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation* 14(11):2531–2560, 2002.
- [8] Clark, A., Chalmers, D. The extended mind. *Analysis* 58(1):7–19, 1998.